

BeautifulSoup Documentation Analytics — Final Report

Generated: 2026-07-13 12:46

Target URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

1. Dataset Overview

Dataset	File	Rows	Columns
Sections	`data/processed/sections.csv`	113	7
Links	`data/processed/links.csv`	504	4
Code Examples	`data/processed/code_examples.csv`	220	9

Section columns: section_id, section_level, section_title, section_text, word_count, code_block_count, link_count

Link columns: link_text, href, link_type, section_title

Code example columns: example_id, section_title, code_text, line_count, contains_find_all, contains_find, contains_select, contains_get_text, contains_requests

2. Scraping Method

The system uses the following Python libraries to collect and parse the BeautifulSoup documentation:

1. **requests** — Sends an HTTP GET request to the target URL and retrieves the raw HTML content. The response status code is checked via `raise_for_status()` to ensure a successful download.
2. **BeautifulSoup4** — Parses the raw HTML into a navigable DOM tree using Python's built-in `html.parser`.
3. **pandas** — Structures the extracted data into DataFrames and exports them to CSV files for downstream analysis.
4. **numpy** — Supports numerical operations during analytics.
5. **matplotlib** — Generates the data visualization charts.

Pipeline flow:

Download HTML → Parse with BeautifulSoup → Extract Sections / Links / Content
→ Save to CSV → Analyze with Pandas+NumPy → Visualize → Generate Report

3. Extracted Data Summary

3.1 Sections

Metric	Value
Total sections	113
Section levels	h1, h2, h3
Average word count	363.3
Total code blocks	1751
Total links in sections	804

3.2 Links

Link Type	Count
internal_anchor	470
documentation_link	15
external_link	17
image_link	0
empty_or_invalid	2

3.3 Code Examples

Metric	Value
Total code examples	220
Average line count	6.34
Max line count	37
Examples using <code>`find_all()`</code>	41
Examples using <code>`get_text()`</code>	4

4. Analysis Results

4.1 Required Questions

#	Question	Answer
Q1	How many sections are in the documentation?	113
Q2	Which section has the highest word count?	"Searching the tree ¶" (4711 words)

Q3	Which section contains the most code examples?	"Searching the tree ¶" (208 blocks)
Q4	Which section contains the most links?	"Table of Contents" (127 links)
Q6	How many internal and external links?	Internal: 470, Doc: 15, External: 17
Q7	Code examples using <code>find_all()</code> ?	41
Q8	Code examples using <code>get_text()</code> ?	4

4.2 Top 10 Technical Keywords (Q5)

Rank	Keyword	Frequency
1	soup	926
2	tag	746
3	class	511
4	html	420
5	example	419
6	string	387
7	com	383
8	href	378
9	http	363
10	beautiful	351

4.3 Additional Questions

Q9: Code line statistics

- Average lines per code example: 6.34
- Longest code example: "Parsing only part of a document ¶" (37 lines)

Q10: Statistics by heading level

Level	Avg Word Count	Avg Code Blocks	Avg Links
H1	1011.2	40.9	18.6
H2	295.7	12.0	4.8
H3	153.8	8.3	4.7

5. Charts

The following charts were generated as part of Feature 7 (Data Visualization) and saved to `output/charts/`.

Chart 1: Top 10 Sections by Word Count

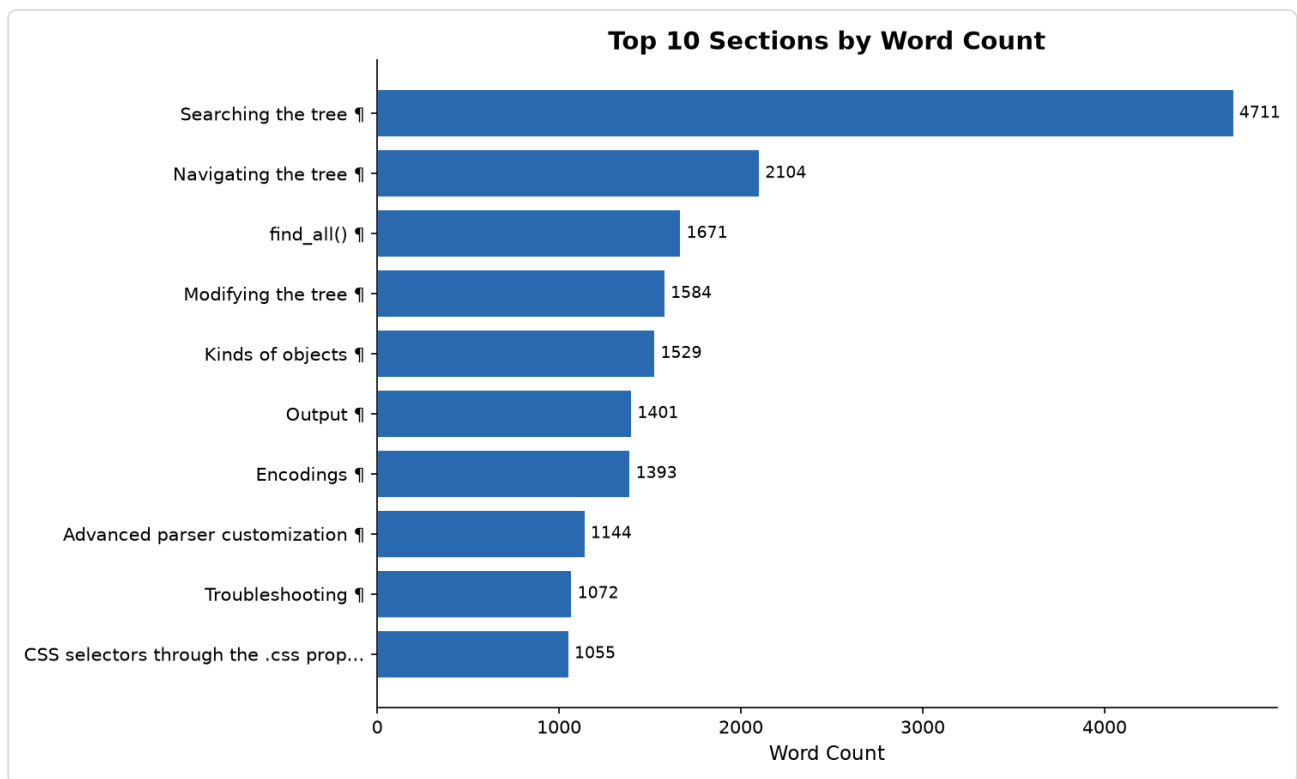


Chart 2: Code Examples by Section

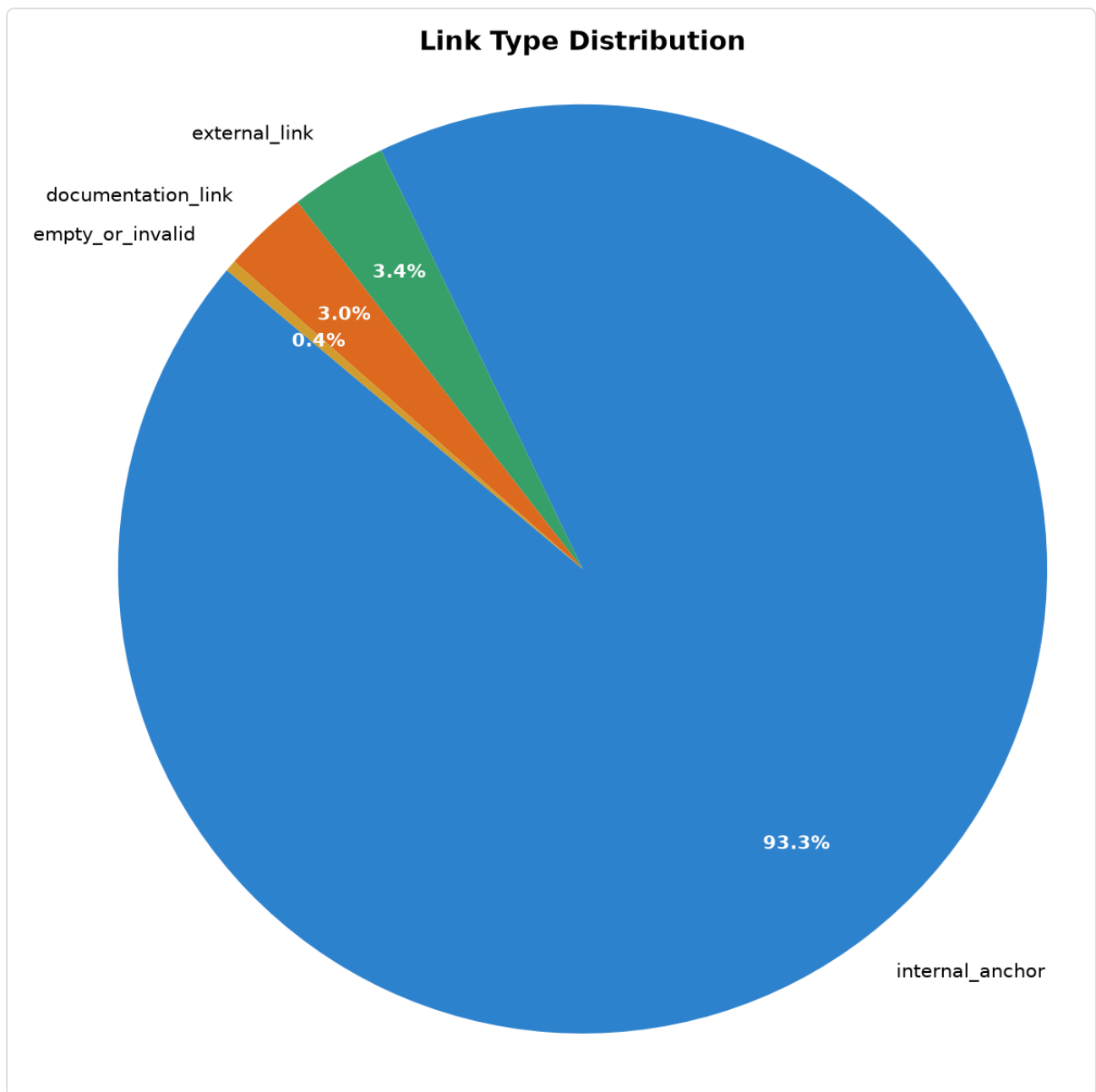
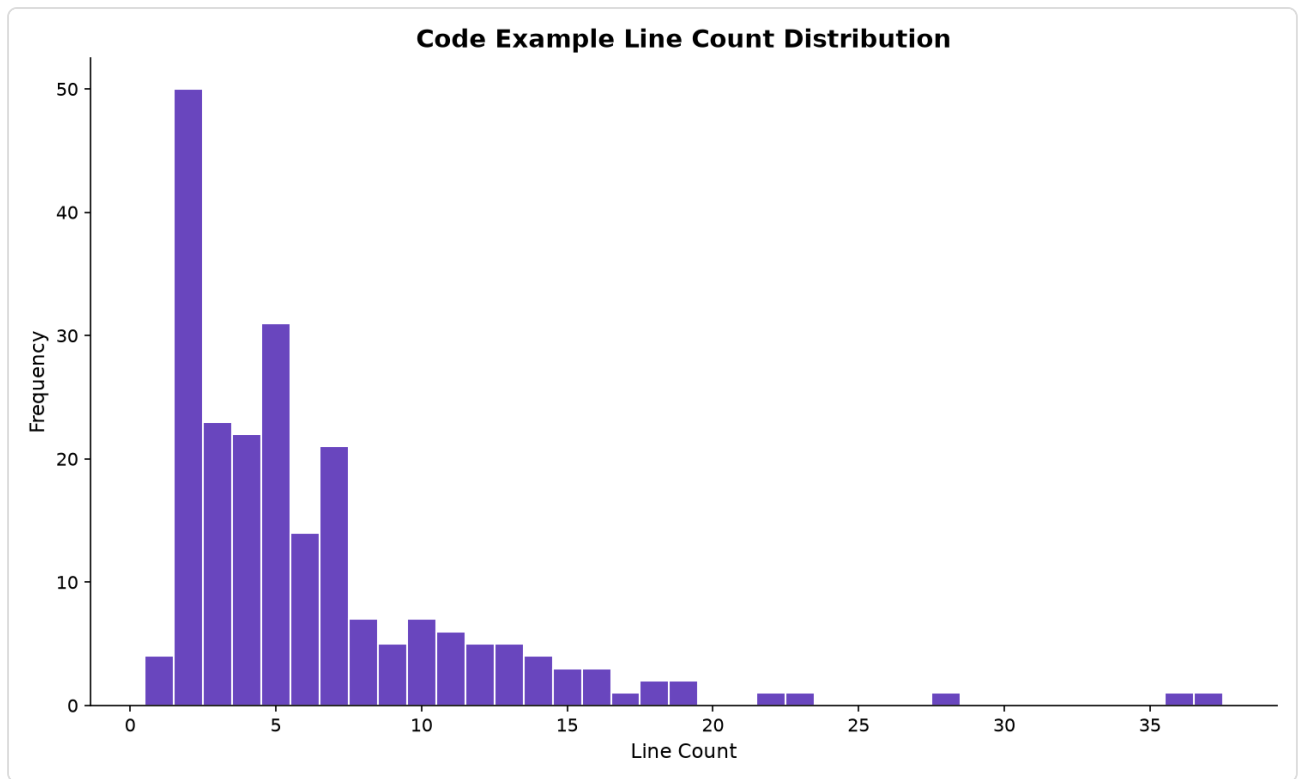


Chart 4: Code Example Line Count Distribution



6. Key Findings

1. The BeautifulSoup documentation contains **113 sections**

across h1, h2, and h3 heading levels.

2. The section with the most content is

"Searching the tree 1"

with 4711 words.

3. **"Searching the tree 1"** has the most code examples

(208 blocks), making it the

most code-heavy section.

4. **"Table of Contents"** contains the most links

(127), indicating it is the most

reference-rich section.

5. The most frequent technical keyword is

"soup"

(926 occurrences).

6. Link classification shows 470 internal anchors,

15 documentation links, and 17

external links.

7. 41 code examples use `find_all()` and

4 use `get_text()`, confirming these are the

most demonstrated BeautifulSoup APIs.

8. The average code example is 6.3 lines long,

with the longest at 37 lines in

"Parsing only part of a document ¶".

7. Limitations

1. **Single-page scope:** Only the main BeautifulSoup documentation page was

scraped; sub-pages and linked external resources were not crawled.

2. **Parser dependency:** Results may vary slightly depending on the HTML parser used (`html.parser` vs `lxml` vs `html5lib`).

3. **Dynamic content:** JavaScript-rendered content is not captured by `requests` ; only static HTML is analyzed.

4. **Keyword filtering:** Stopword list is manually curated and may not capture all domain-irrelevant terms.

5. **Code example detection:** Only

blocks are treated as code

examples; inline snippets within paragraphs are not counted

as separate examples.

8. Conclusion

This project successfully built an end-to-end Python data pipeline that

collects, parses, extracts, analyzes, and visualizes the BeautifulSoup
official documentation. The system leverages requests for web scraping,
BeautifulSoup4 for HTML parsing, pandas and numpy for data analysis,
and matplotlib for visualization.

The analysis reveals that the documentation is well-structured with
113 sections, rich in code examples
(220 total), and heavily cross-referenced with
504 links. The most frequently demonstrated APIs are
find_all() and get_text(), which aligns with BeautifulSoup's core
usage patterns.

The generated charts and this report provide a clear, reproducible summary of the documentation's structure and content, demonstrating the team's ability to apply data engineering techniques to real-world technical documentation.

Report generated by BeautifulSoup Documentation Analytics System.